

CIS 022 day 2

getting started with c++

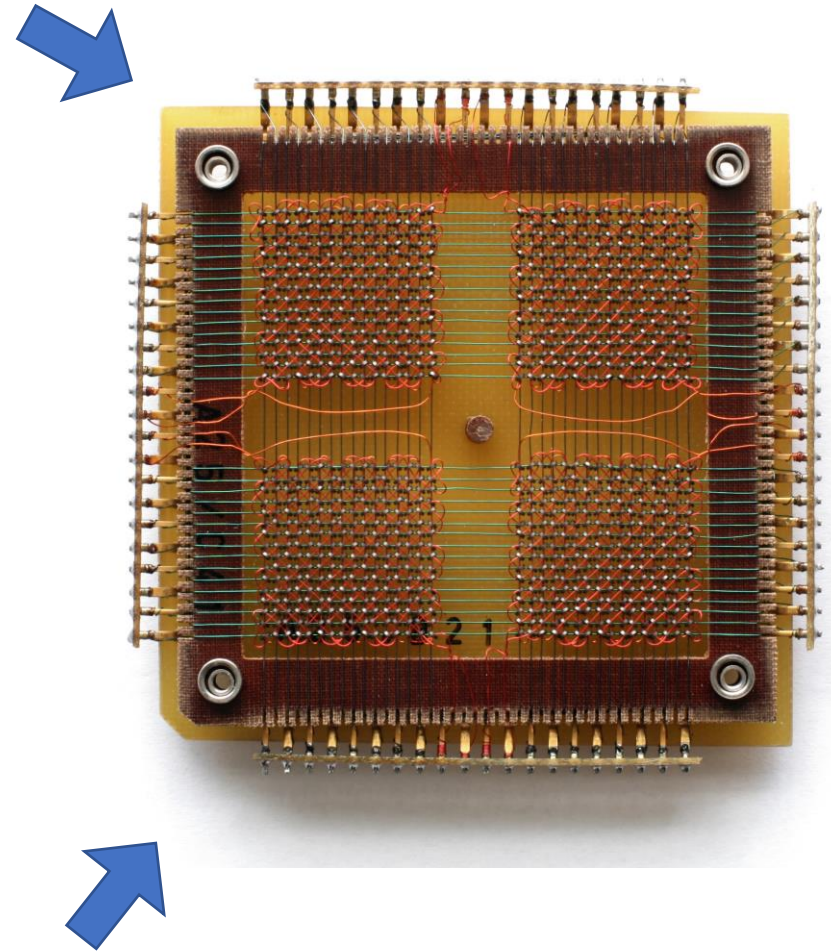
Plan for today

- Most of today's material will be review for people who've worked in C++ before
- (and really, it will be *largely* review for people who've worked in Java or Python.)
- There's going to be some stuff toward the end that might be entirely new to you, though.

memory

this is what memory looked like in 1970s computers... the last time memory was big enough to see with the naked eye

- The thing you will be doing most often when you write programs is writing to memory and reading from memory.
- (if we were in an intro class we'd talk about how CPUs work right now. This is not an intro class, so we're just going to discuss memory.)

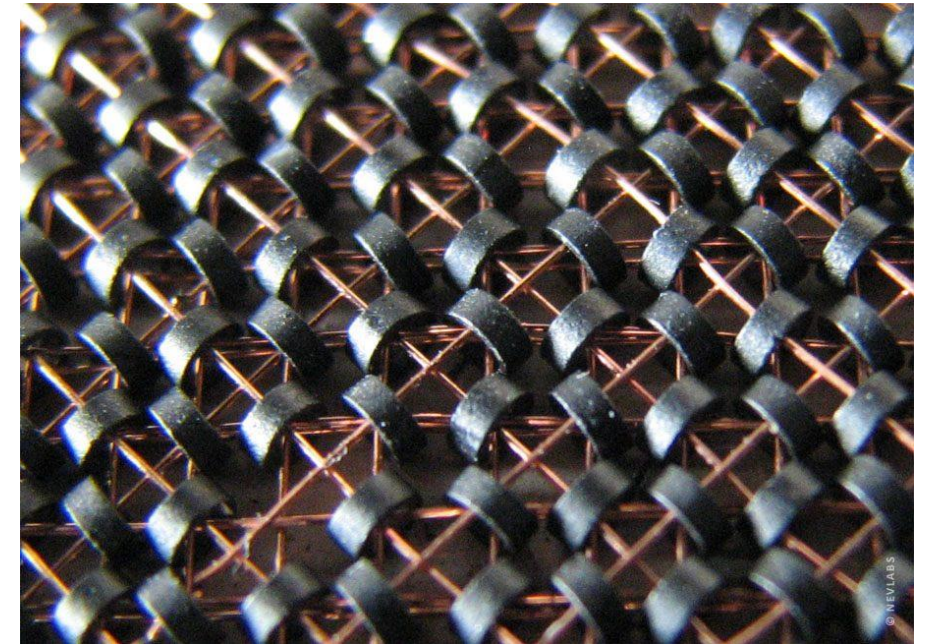


this board holds about 768 bits (I counted!)

memory

- The thing you will be doing most often when you write programs is writing to memory and reading from memory.
- (if we were in an intro class we'd talk about how CPUs work right now. This is not an intro class, so we're just going to discuss memory.)

here's a zoomed-in picture of 1970s memory – each one of these metal disks is a bit. it's a 1 if it's magnetized and a 0 if it's not.

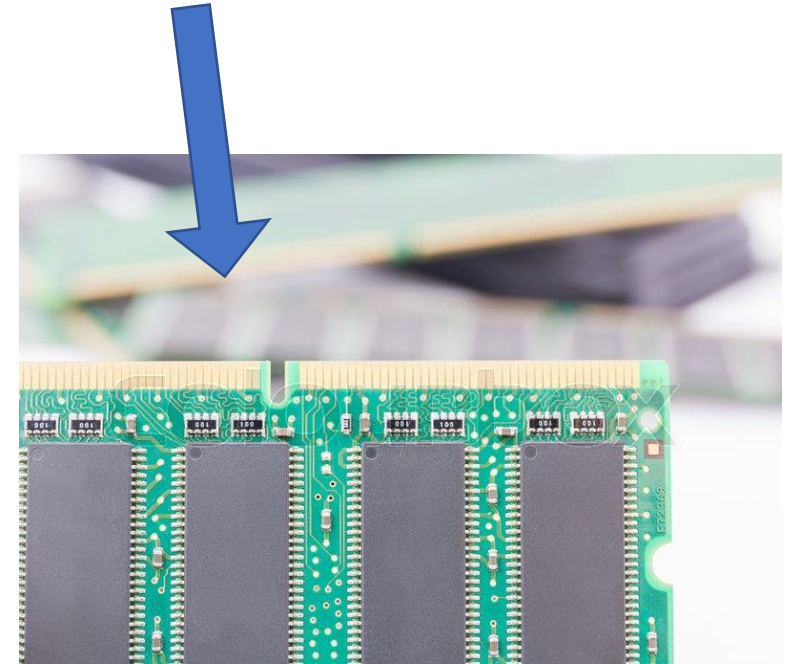


(this is about 72 bits)

memory

- The thing you will be doing most often when you write programs is writing to memory and reading from memory.
- (if we were in an intro class we'd talk about how CPUs work right now. This is not an intro class, so we're just going to discuss memory.)

here's a modern memory chip – this board contains billions of bits, all of which are far too small to see



this board holds about 128,000,000,000 bits.

memory – an analogy

- You can think of your computer's memory as something like a gigantic spreadsheet.
- Each cell on the spreadsheet (or each *address* in memory) can hold a piece of data – a number, a string of letters, a true-or-false value, etc.



true-or-false values
are called *booleans*

	A	B	C	D	E	F	G	H
1	STUDENT	MULTIPLE CHOICE	q1	q2	q3	q4	extra credit	TOTAL
2	NAMES	33	9	9	10	12	0	73
3	DELETED	33	15	14	15	15	5	97
4	BECAUSE	39	15	10	15	15	10	104
5	IT	33	13	8	15	15	5	89
6	WOULD	24	15	15	1	5	10	70
7	BE	24	7	0	12	14	5	62
8	RUDE	24	15	15	15	15	0	84
9	TO	30	10	15	15	15	0	85
10	SHOW	18	15	15	5	10	3	66
11	THEM	27	15	5	10	15	5	77
12	TO	36	15	15	15	15	10	106
13	YOU	30	14	15	15	14	0	88

memory – an analogy

- You can think of your computer's memory as something like a gigantic spreadsheet.
- Each cell on the spreadsheet (or each *address* in memory) can hold a piece of data – a number, a string of letters, a true-or-false value, etc.

we can think of this as the *string* "extra credit" stored at *address* G1 on the spreadsheet



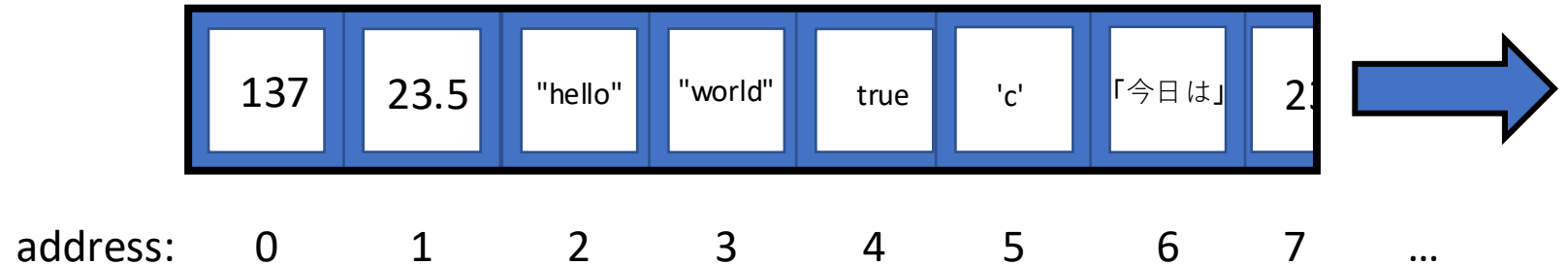
	A	B	C	D	E	F	G	H
1	STUDENT	MULTIPLE CHOICE	q1	q2	q3	q4	extra credit	TOTAL
2	NAMES	33	9		9	10	12	73
3	DELETED	33	15		14	15	15	97
4	BECAUSE	39	15		10	15	15	104
5	IT	33	13		8	15	15	89
6	WOULD	24	15		15	1	5	70
7	BE	24	7		0	12	14	62
8	RUDE	24	15		15	15	15	84
9	TO	30	10		15	15	15	85
10	SHOW	18	15		15	5	10	66
11	THEM	27	15		5	10	15	77
12	TO	36	15		15	15	15	106
13	YOU	30	14		15	15	14	88



We can think of this as the *integer* 30 stored at the *address* B13 on the spreadsheet

memory – another analogy

- Addresses on a spreadsheet is two dimensional... but on your computer, memory addresses are sequential, in a line
- You can think of your computer's memory as an *extremely* long strip of paper, divided up into little boxes. Each box can hold one piece of data (a number, a string, a boolean value, etc.)
- The first box is at address 0, the second is at address 1, the third box is at address 2, and so forth... all the way to address 128,000,000,000 (or whatever, depending on how much memory your computer has.)

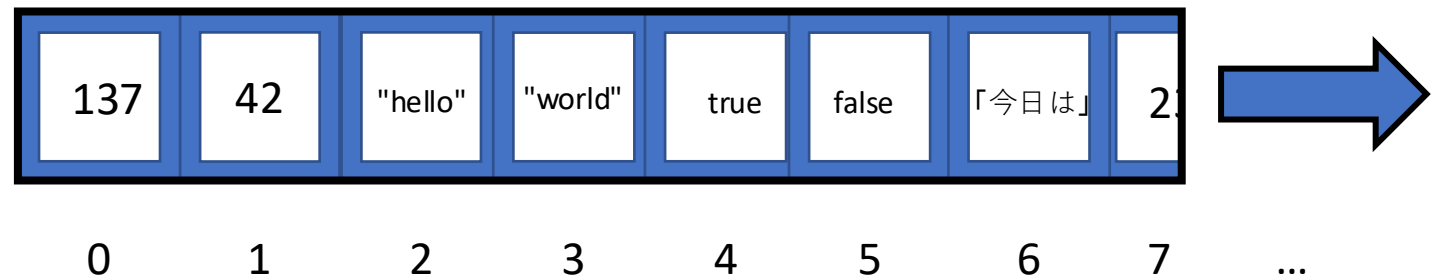
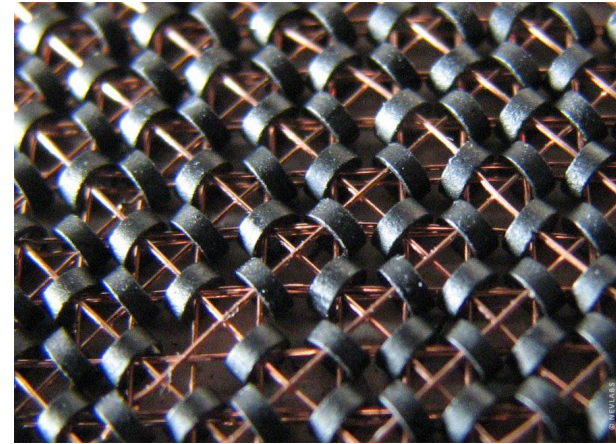


* this is a simplification for the purpose of this class. if you want to know the details, take systems programming

FROM BITS TO BOXES

Now you may be thinking:

- Bits are 0s and 1s
- Boxes are numbers, letters, strings of letters... how do you turn 0s and 1s into numbers and words?

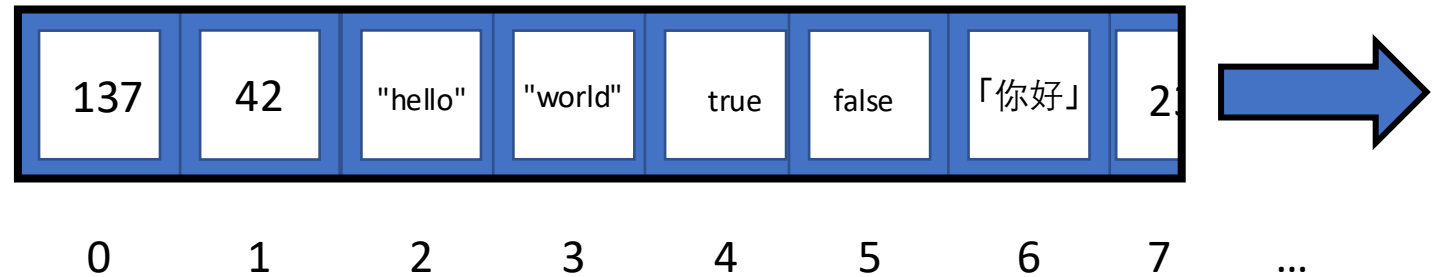
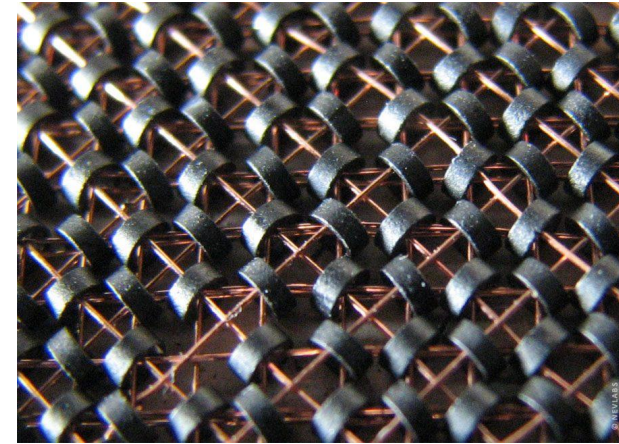


HOW BINARY WORKS

- The trick is binary notation -- counting in base two

BASE TWO	BASE TEN
0	0
1	1
10	2
11	3
100	4
101	5
110	6
111	7
1000	8
1001	9
1010	10

you can represent
the number ten
with 4 bits

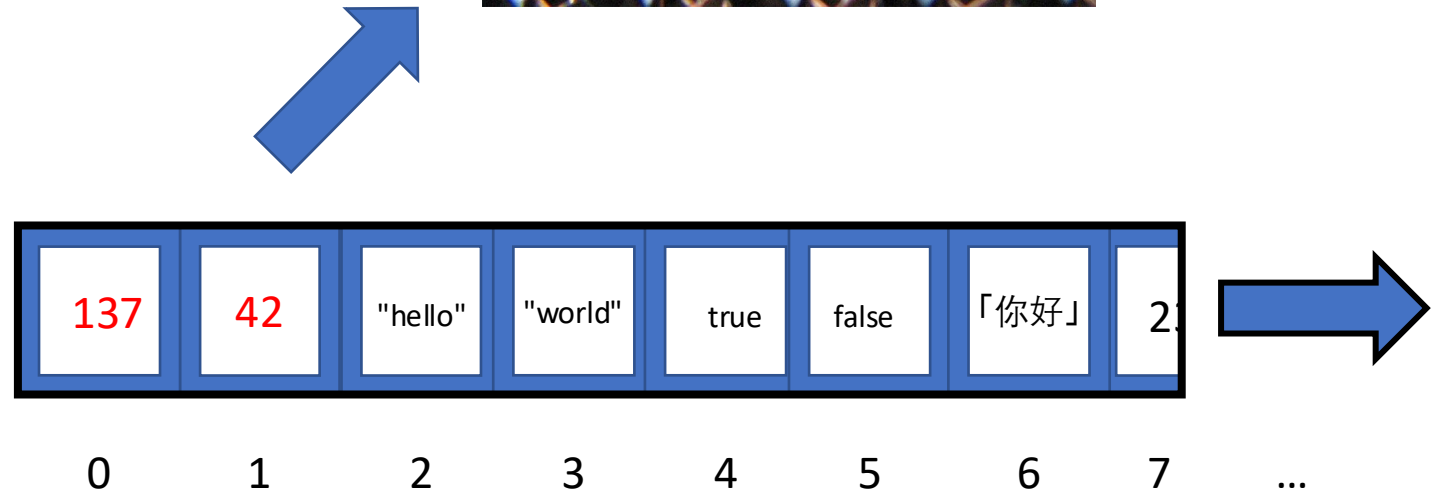
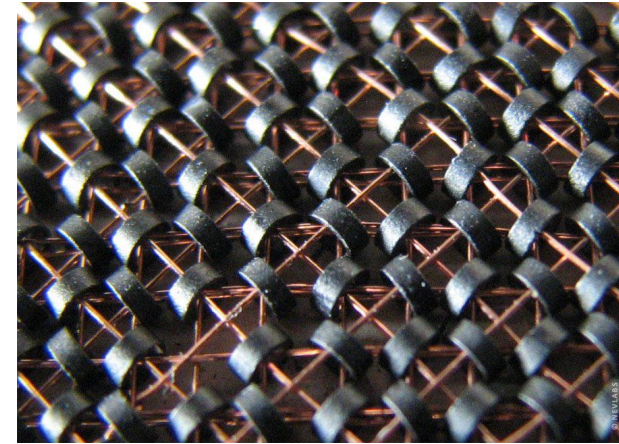


HOW BINARY WORKS

- The trick is binary notation -- counting in base two

BASE TWO	BASE TEN
0	0
1	1
10	2
11	3
100	4
101	5
110	6
111	7
1000	8
1001	9
1010	10

that's how you can represent numbers like 137 or 42 in a computer – by using bits to write down the base two version.



HOW BINARY WORKS

BASE TWO	BASE TEN
0	0
1	1
10	2
11	3
100	4
101	5
110	6
111	7
1000	8
1001	9
1010	10

you can represent
the number ten
with 4 bits



QUICK QUIZ:

you can represent 8 different numbers with 3 bits (every number from 0 to 7)

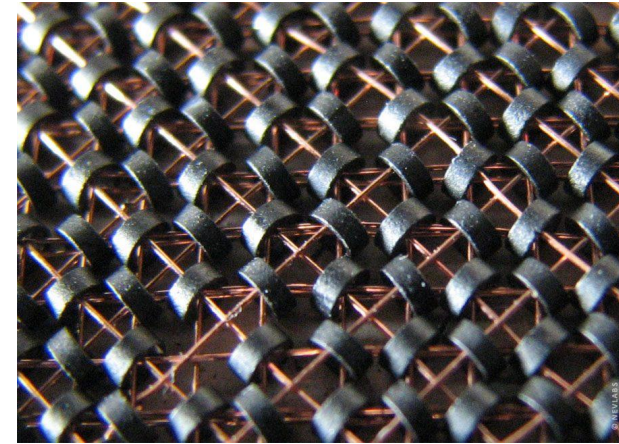
how many different numbers can you represent with four bits?

what about five bits?

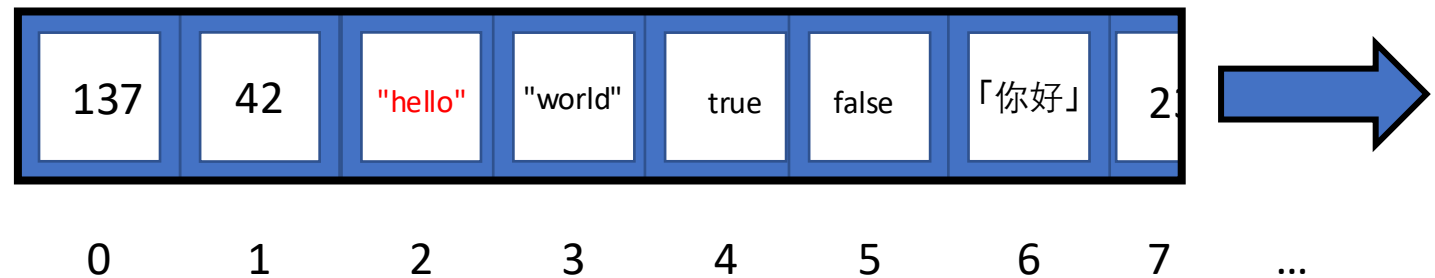
(spend a minute or so working it out on paper if you need to)

FROM NUMBERS TO LETTERS

- Okay, that's how we represent numbers. But how do we represent letters and words?



Note: obviously these boxes aren't all the same size – different types of variables take different amounts of space. An integer generally takes 4 bytes of space (32 bits). A single character takes 1 byte. A string takes up (at least) the number of bytes required to store the number of characters in the string.



FROM NUMBERS TO LETTERS

- Here's the trick: there's an arbitrary standard called ASCII (short for "American Standard Code for Information Interchange" that tells how to translate from letters to numbers.
- ASCII uses 7 bits to store each letter, plus another bit for error detection (we'll talk about this later on when we talk about networking). How many different characters can you represent in 7 bits?
- 8 bits is called a byte. A single character is 1 byte. A word (or a sentence, or a paragraph, or a novel...) is a bunch of single characters put together in a *string*.
- See www.asciitable.com

the number 65
means capital A.

the number 113
means lowercase q

Dec	Char	Dec	Char
64	@	96	`
65	A	97	a
66	B	98	b
67	C	99	c
68	D	100	d
69	E	101	e
70	F	102	f
71	G	103	g
72	H	104	h
73	I	105	i
74	J	106	j
75	K	107	k
76	L	108	l
77	M	109	m
78	N	110	n
79	O	111	o
80	P	112	p
81	Q	113	q
82	R	114	r
83	S	115	s
84	T	116	t
85	U	117	u
86	V	118	v
87	W	119	w
88	X	120	x
89	Y	121	y
90	Z	122	z
91	[	123	{
92	\	124	
93	]	125	}
94	^	126	~
--	--	---	---

WHAT ABOUT 你好 and مرحبا and こんにちは?

- ASCII (8 bits) can only represent characters in the Latin character set
- What about all the other character sets in the world?
- Unicode is a character encoding that uses up to 32 bits (4 bytes) to represent each character.
- Because it's 4 bytes wide, Unicode can represent 4,294,967,296 different characters – 2^{32} .
- (Nearly) every language in the world can be represented in Unicode
- We'll be (mostly) just using ASCII characters in this class, because our programming tools make it easier to work in ASCII than in Unicode ☹️



it's actually a little less than this –
to find out why, take systems
programming

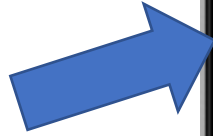
hello world in C++

because some folks learned to code in Python and Java

SAMPLE PROGRAM

- Here's a sample program:

you'll put this at the top of nearly every program in this class. it imports the code that lets you use input and output streams



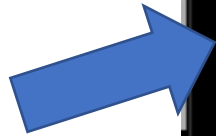
```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "The magic number is: " << magicNumber << endl;
9
10    return 0;
11 }
12
13
```

SAMPLE PROGRAM

- Here's a sample program:

you'll put this at the top of nearly every program in this class

(in later C++ courses, and in industry, doing things this way is often considered bad practice. It simplifies the syntax enough that it's worth doing in this course, though.



```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "The magic number is: " << magicNumber << endl;
9
10    return 0;
11 }
12
13
```

SAMPLE PROGRAM

- Here's a sample program:

int main()

is the function
where program
execution starts.



```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "The magic number is: " << magicNumber << endl;
9
10    return 0;
11 }
12
13
```

For python programmers: the code in int main() is equivalent to the code you call from

```
if __name__ == "__main__":
```

SAMPLE PROGRAM

- Here's a sample program:

int main()

is the function
where program
execution starts.



```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "The magic number is: " << magicNumber << endl;
9
10    return 0;
11 }
12
13
```

For Java programmers: this is equivalent to

```
public static void main(String[] args)
```

SAMPLE PROGRAM

- Here's a sample program:

int, here, is the
return type of
the function.
When this
function ends, it
passes back an
int: an integer
value



```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "The magic number is: " << magicNumber << endl;
9
10    return 0;
11 }
12
13
```

SAMPLE PROGRAM

- Here's a sample program:

int, here, is the *return type* of the function. When this function ends, it passes back an int: an integer value.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "The magic number is: " << magicNumber << endl;
9
10    return 0;
11
12
13
```



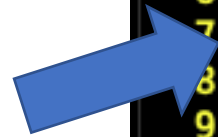
By convention, main() always returns 0.

Why? When the program ends, the operating system can do different things based on what value your program returns. If your program runs successfully and gets to the end of main(), it will return 0 – which (again by convention) indicates that your program has run successfully. If something causes your program *not* to run successfully – say that it crashes – it will return some value other than 0.

SAMPLE PROGRAM

- Here's a sample program:

Here I'm
*declaring a
variable*. In C++,
unlike in Python,
you must declare
a variable before
you use it.



```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "The magic number is: " << magicNumber << endl;
9
10    return 0;
11 }
12
13
```

SAMPLE PROGRAM

- Here's a sample program:

This is the type
of the variable –
an int, same as
main()'s return
type



```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int magicNumber = 137;
7     cout << "The magic number is: " << magicNumber << endl;
8
9     return 0;
10 }
11
12
13
```

SAMPLE PROGRAM

- Here's a sample program:

And here I'm
initializing the
variable – setting
it equal to 137

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "magic number is: " << magicNumber << endl;
9
10    return 0;
11 }
12
13
```



SAMPLE PROGRAM

- Here's a sample program:

As in most languages, you assign a value to a variable by using a single = sign. If you want to test whether two variables are equal, use == instead.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "magic number is: " << magicNumber << endl;
9
10    return 0;
11 }
12
13
```



SAMPLE PROGRAM

- Here's a sample program:

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "The magic number is: " << magicNumber << endl;
9
10    return 0;
11 }
12
13
```

Here's how you
do output in C++

cout is short for **console out**. It means "print to the screen." cout is a "stream" – for now, just think of a stream as something you can write to or read from.

<< is the **stream insertion operator**. It means "put everything on the right hand side of << into the stream on the left hand side of <<

SAMPLE PROGRAM

- Here's a sample program:

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "The magic number is: " << magicNumber << endl;
9
10    return 0;
11 }
12
13
```

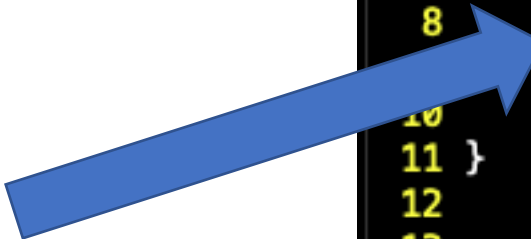
Here's how you
do output in C++

endl means "end line" – this inserts a newline character into the stream

SAMPLE PROGRAM

- Here's a sample program:

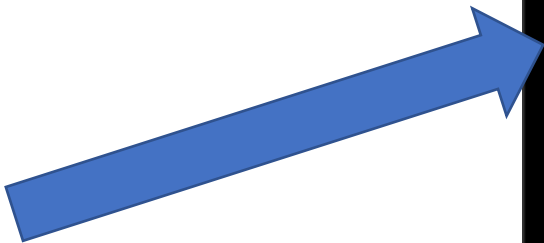
```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int magicNumber = 137;
8     cout << "The magic number is: " << magicNumber << endl;
9
10    return 0;
11 }
12
13
```



If we *didn't* include using namespace std at the top of this file, we'd have to use somewhat more complicated syntax here:

SAMPLE PROGRAM

- Here's a sample program:



```
1 #include <iostream>
2
3
4 int main(){
5
6     int magicNumber = 137;
7     std::cout << "The magic number is: " << magicNumber << std::endl;
8
9     return 0;
10 }
11
```

If we *didn't* include using namespace std at the top of this file, we'd have to use somewhat more complicated syntax here:

SAMPLE PROGRAM

- Karel (deliberately) doesn't include variables.
- (that's what makes the puzzles interesting)
- What does it mean to "declare a variable" in C++?
- It means to *set aside space in memory* for that type of variable.
- So this command means set aside space in memory for an int (a whole number; something without a decimal point) and then stores the number 137 there.

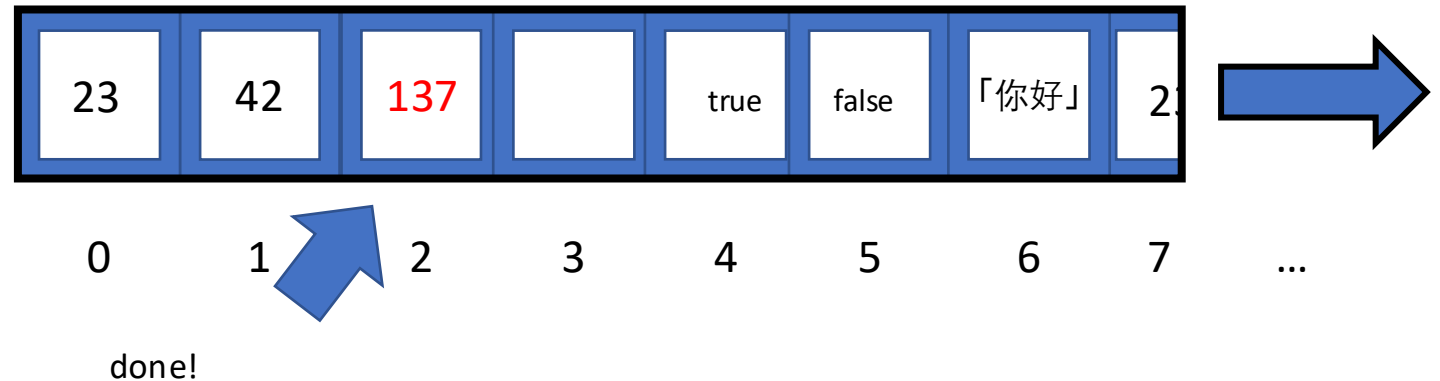


```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int magicNumber = 137;
7     cout << "The magic number is: " << magicNumber << endl;
8
9 }
10
11
```

sample PROGRAM

- When the compiler sees this command, it grabs a box that isn't storing anything yet (one that's the appropriate size for an int) and then stores the number 137 there.

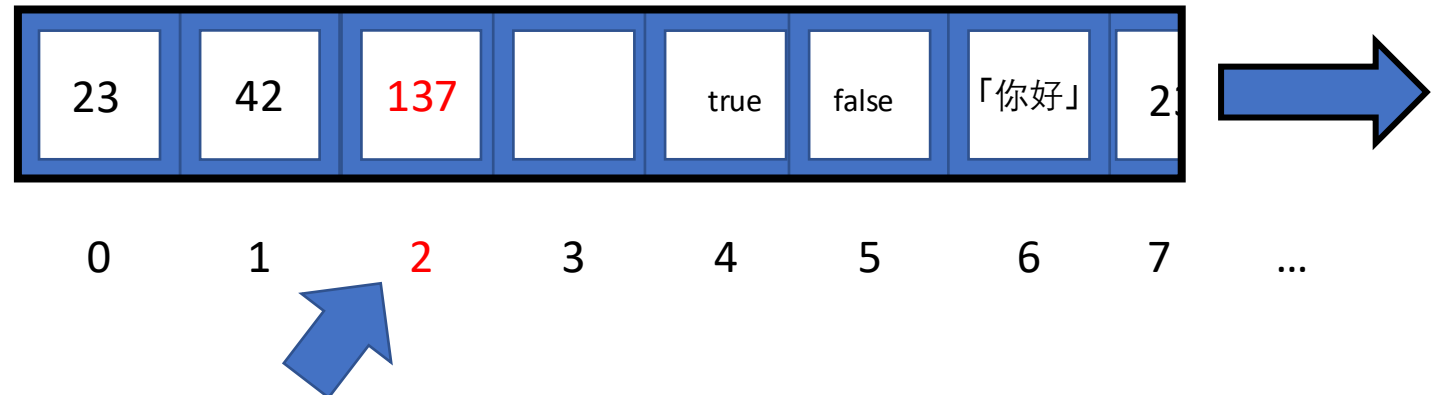
```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int magicNumber = 137;
7     cout << "The magic number is: " << magicNumber << endl;
8
9 }
10
11
```



sample PROGRAM

- When the compiler sees this command, it grabs a box that isn't storing anything yet (one that's the appropriate size for an int) and then stores the number 137 there.
- It would be pretty inconvenient to have to refer to this variable by its address, so we give it a name: `magicNumber`.
- Now whenever we want to refer to the variable at address 2 (or whatever address the compiler assigns), we can use its name: `magicNumber`

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int magicNumber = 137;
7     cout << "The magic number is: " << magicNumber << endl;
8
9 }
10
11
```



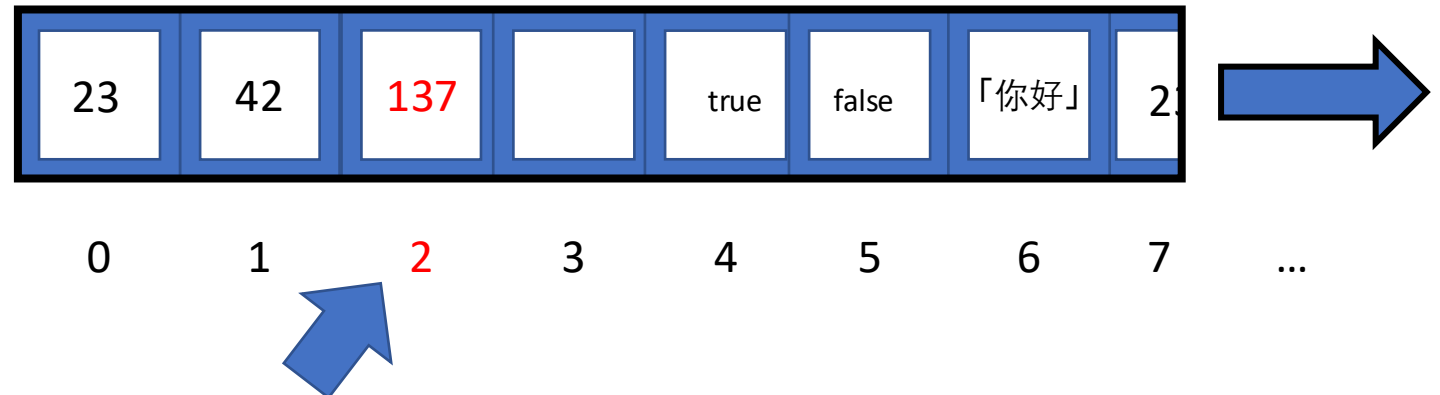
sample PROGRAM (syntax note)

- Syntax note: remember how in Karel when we defined a function we used the **function** keyword, followed by the function's name:

```
function safeMove() {  
    if(frontIsClear()){  
        move();  
    }  
}
```

- But when we used the function we just used its name:
 function main(){
 safeMove();
 }

```
1 #include <iostream>  
2  
3 using namespace std;  
4  
5 int main(){  
6     int magicNumber = 137;  
7     cout << "The magic number is: " << magicNumber << endl;  
8  
9 }  
10  
11
```



sample PROGRAM (syntax note)

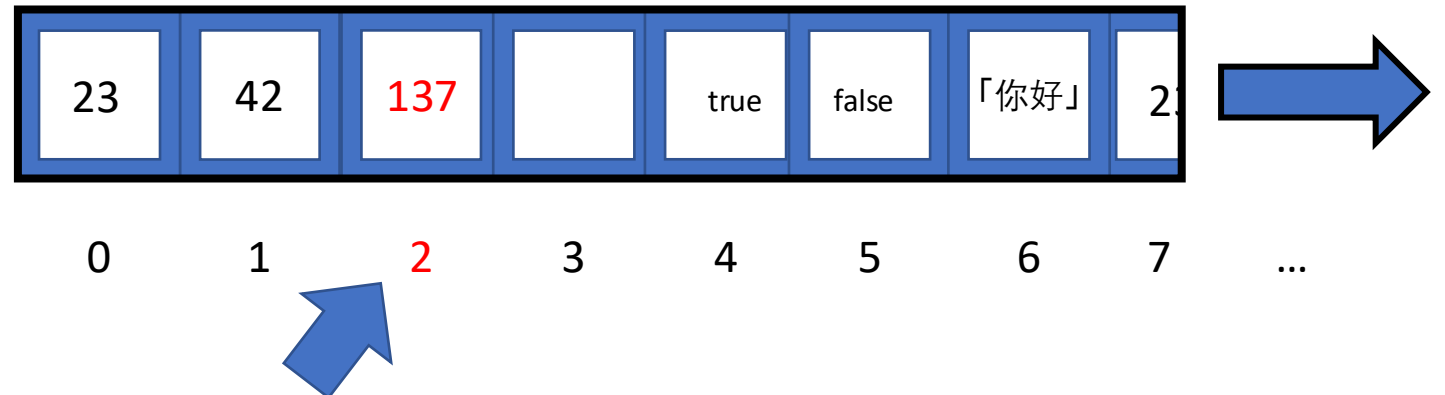
- Syntax note:
- C++ works the same:

When you *define* a variable, you use the *type* of the variable, followed by the variable's name

- (this tells the compiler how much space in memory it needs to set aside for the variable)

- In this case, the type is **int**

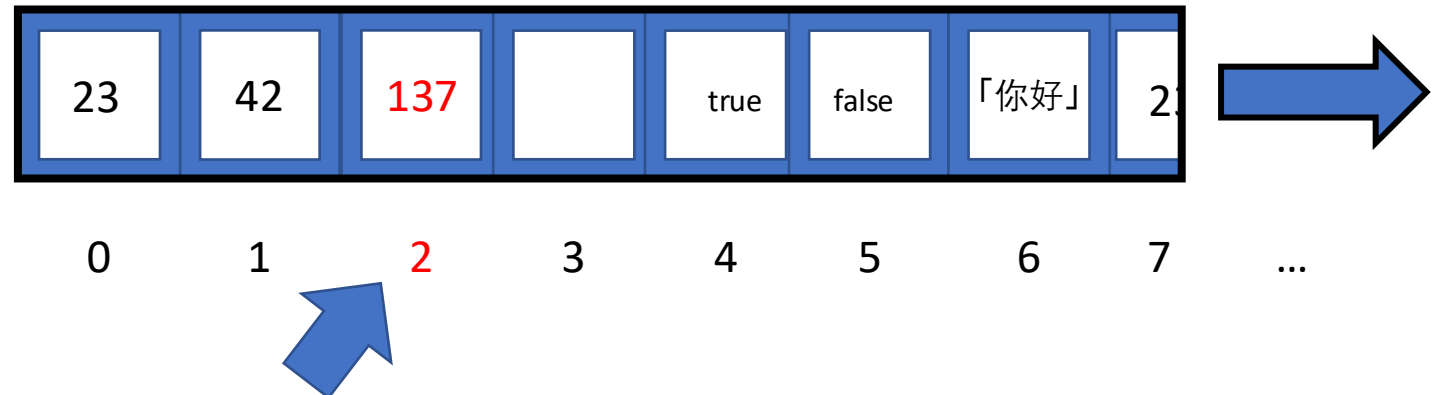
```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int magicNumber = 137;
7     cout << "The magic number is: " << magicNumber << endl;
8
9 }
10
11
```



sample PROGRAM (syntax note)

- Syntax note:
- But when you *use* the variable, you just give its name.
- Using a variable means "go fetch the thing that's in the box you've set aside for this name."

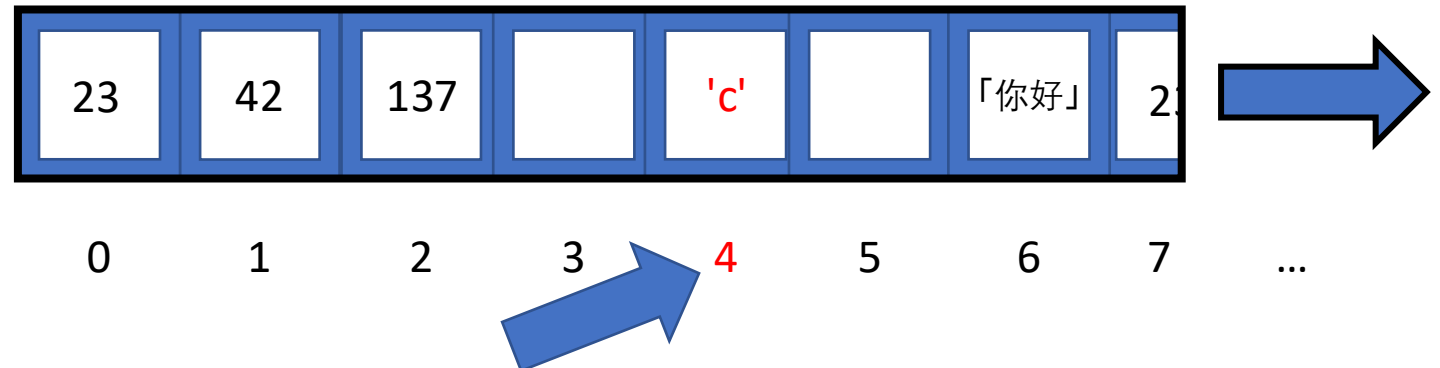
```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int magicNumber = 137;
7     cout << "The magic number is: " << magicNumber << endl;
8
9 }
10
11
```



sample PROGRAM (now with characters)

- Okay, but what if we want to store something other than numbers?
- If we want to store a single character, we use the **char** type
- This command means "set aside space in memory for the character 'c'"
- You might remember that characters are just numbers
- So when you use the **char** type, it also means "when you use this type later on, look up its value in the ASCII table and use the letter associated with that number"

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int magicNumber = 137;
7     char theLanguage = 'c';
8     cout << theLanguage << "++ is fun!" << endl;
9
10 }
11
12
```

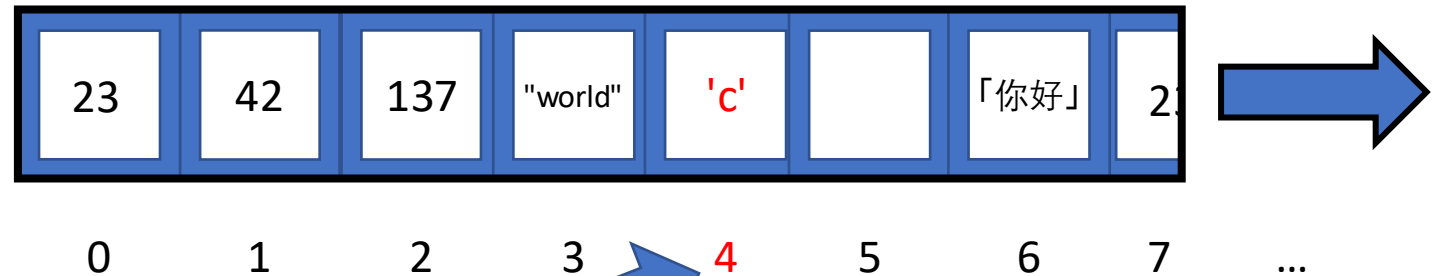


this looks like a good space!

sample PROGRAM (now with characters)

Dec	Char	Dec	Char
64	@	96	`
65	A	97	a
66	B	98	b
67	C	99	c
68	D	100	d
69	E	101	e
70	F	102	f
71	G	103	g
72	H	104	h
73	I	105	i
74	J	106	j
75	K	107	k
76	L	108	l
77	M	109	m
78	N	110	n
79	O	111	o
80	P	112	p
81	Q	113	q
82	R	114	r
83	S	115	s
84	T	116	t
85	U	117	u
86	V	118	v
87	W	119	w
88	X	120	x
89	Y	121	y
90	Z	122	z
91	[	123	{
92	\	124	
93	]	125	}
94	^	126	~
--		---	

so the value stored at address 4
is really 99... but whenever you
print it out it's automatically
converted to the letter c



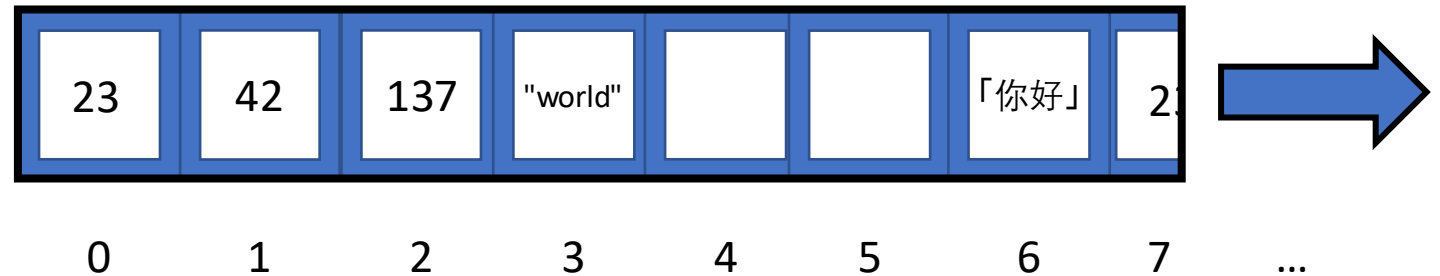
this looks like a good space!

sample PROGRAM (now with strings)

- Okay, but what if we want to store whole words?
- we have to include the *string library*, like so:



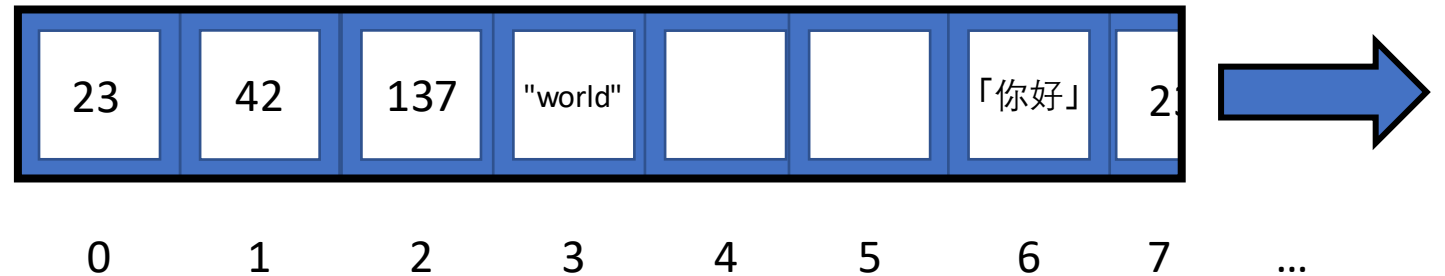
```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 int main(){
7     int magicNumber = 137;
8     string greeting = "Hello!";
9     cout << greeting << endl;
10    cout << "The magic number is: " << magicNumber << endl;
11
12 }
13
14
```



sample PROGRAM (now with strings)

- Okay, but what if we want to store whole words?
- We have to include the *string library*, like so:
- Then we declare a variable of type "string." This sets aside space in memory for a whole bunch of characters. We name this string **greeting**, and store the value "Hello!" there.

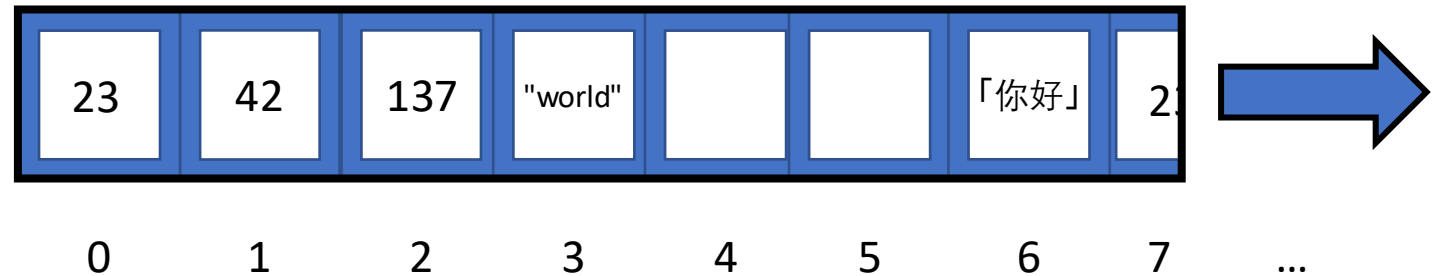
```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 int main(){
7     int magicNumber = 137;
8     string greeting = "Hello!";
9     cout << greeting << endl;
10    cout << "The magic number is: " << magicNumber << endl;
11
12 }
13
14
```



sample PROGRAM (now with strings)

- There is another, older way to store strings in c++. These are called c-strings, and we'll talk about them when we have to.
- (long story short: c-strings are just *arrays of characters*. They're slightly more efficient than c++ style strings, but much more annoying to work with)
- (we'll talk about arrays in a few slides)

```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 int main(){
7     int magicNumber = 137;
8     string greeting = "Hello!";
9     cout << greeting << endl;
10    cout << "The magic number is: " << magicNumber << endl;
11
12 }
13
14
```



sample PROGRAM (now with input)

- Okay, you know how to write data to the console. How do you read data from the keyboard?
- You do this using `cin` (short for **console in**) and `>>` (the *stream extraction operator*)

```
Please enter your name: ben
Hello, ben
Bens-MacBook-Air-2:testbed benallen$
```

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     string userName;
8     cout << "Please enter your name: ";
9
10    cin >> userName;
11
12    cout << "Hello, " << userName << endl;
13
14    return 0;
15 }
```

sample PROGRAM (now with input)

- You can read multiple values at once by stringing together multiple copies of the stream extraction operator

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int number1, number2;
7
8     cout << "Please enter two integers: ";
9
10    cin >> number1 >> number2;
11
12    cout << number1 << " times " << number2 << " equals "
13        << number1 * number2 << endl;
14
15    return 0;
16 }
17 }
```

```
Bens-MacBook-Air-2:testbed benallen$ ./numbers
Please enter two integers: 3 5
3 times 5 equals 15
Bens-MacBook-Air-2:testbed benallen$
```

Floating-Point Data Types (1 of 3)

- C++ uses scientific notation to represent real numbers (floating-point notation)

Decimal Number	Scientific Notation	C++ Floating-Point Notation
75.924	$7.5924 * 10^1$	7.592400E1
0.18	$1.8 * 10^{-1}$	1.800000E-1
0.0000453	$4.53 * 10^{-5}$	4.530000E-5
-1.482	$-1.482 * 10^0$	-1.482000E0
7800.0	$7.8 * 10^3$	7.800000E3

* this slide and the next few slides are stolen from a textbook
apologies if they're boring and/or ugly

Floating-Point Data Types (2 of 3)

- **float**: represents any real number
 - Range: $-3.4 * 10^{38}$ to $3.4 * 10^{38}$ (four bytes)
- **double**: represents any real number
 - Range: $-1.7 * 10^{308}$ to $1.7 * 10^{308}$ (eight bytes)
- Minimum and maximum values of data types are system dependent

Floating-Point Data Types (3 of 3)

- Maximum number of significant digits (decimal places) for `float` values: 6 or 7
- Maximum number of significant digits for `double`: 15
- Precision: maximum number of significant digits
 - `float` values are called single precision
 - `double` values are called double precision

Data Types, Variables, and Assignment Statements

- To declare a variable, must specify its data type
- Syntax rule to declare a variable is:
 - **dataType identifier;**
- Examples include:

```
int counter;  
double interestRate;  
char grade;
```
- Assignment statement has the form:
 - **variable = expression**
 - Example: **interestRate = 0.05;**

Arithmetic Operators, Operator Precedence, and Expressions (1 of 2)

- C++ arithmetic operators include:
 - + addition
 - subtraction (or negation)
 - * multiplication
 - / division
 - % mod (modulus or remainder)
- +, -, *, and / can be used with integral and floating-point data types
- Modulus (%) can only be used with integral data types

Arithmetic Operators, Operator Precedence, and Expressions (2 of 2)

- When you use `/` with integral data types, the integral result is truncated (no rounding)
- Arithmetic expressions contain values and arithmetic operators
- Operands are the numbers appearing in the expressions
- Operators can be unary (one operand) or binary (two operands)

Order of Precedence

- All operations inside $()$ are evaluated first
- $*$, $/$, and $\%$ are at the same level of precedence and are evaluated next
- $+$ and $-$ have the same level of precedence and are evaluated last
- When operators are on the same level
 - Operations are performed from left to right (associativity)
- $3 * 7 - 6 + 2 * 5 / 4 + 6$ means
 $(((3 * 7) - 6) + ((2 * 5) / 4)) + 6$

Expressions

- Integral expression: all operands are integers
 - Yields an integral result
 - Example: $2 + 3 * 5$
- Floating-point (decimal) expression: all operands are floating-point
 - Yields a floating-point result
 - Example: $12.8 * 17.5 - 34.50$

Mixed Expressions (1 of 2)

- Mixed expression
 - Has operands of different data types
 - Contains integers and floating-point
- Examples of mixed expressions

$2 + 3.5$

$6 / 4 + 3.9$

$5.4 * 2 - 13.6 + 18 / 2$

Mixed Expressions (2 of 2)

- Evaluation rules
 - If operator has same types of operands
 - The operator is evaluated according to the type of the operands
 - If operator has both types of operands
 - Integer is changed to floating-point
 - Operator is evaluated
 - Result is floating-point
 - Entire expression is evaluated according to precedence rules

Type Conversion (Casting) (1 of 2)

- Implicit type coercion: when the value of one type is automatically changed to another type
- Cast operator (also called type conversion or type casting): provides explicit type conversion
 - `static_cast<dataTypeName>(expression)`
- Note: If, later in the class, you find yourself implementing systems where you *have* to use casting, this is generally a sign that your design is not great. Casting is somewhat dangerous – the type system is there to protect you from unpredictable and hard-to-diagnose errors produced by supplying the wrong type of data to functions, and by circumventing the type system through casting you're opening yourself up to all of those errors.

Type Conversion (Casting) (2 of 2)

EXAMPLE 2-9

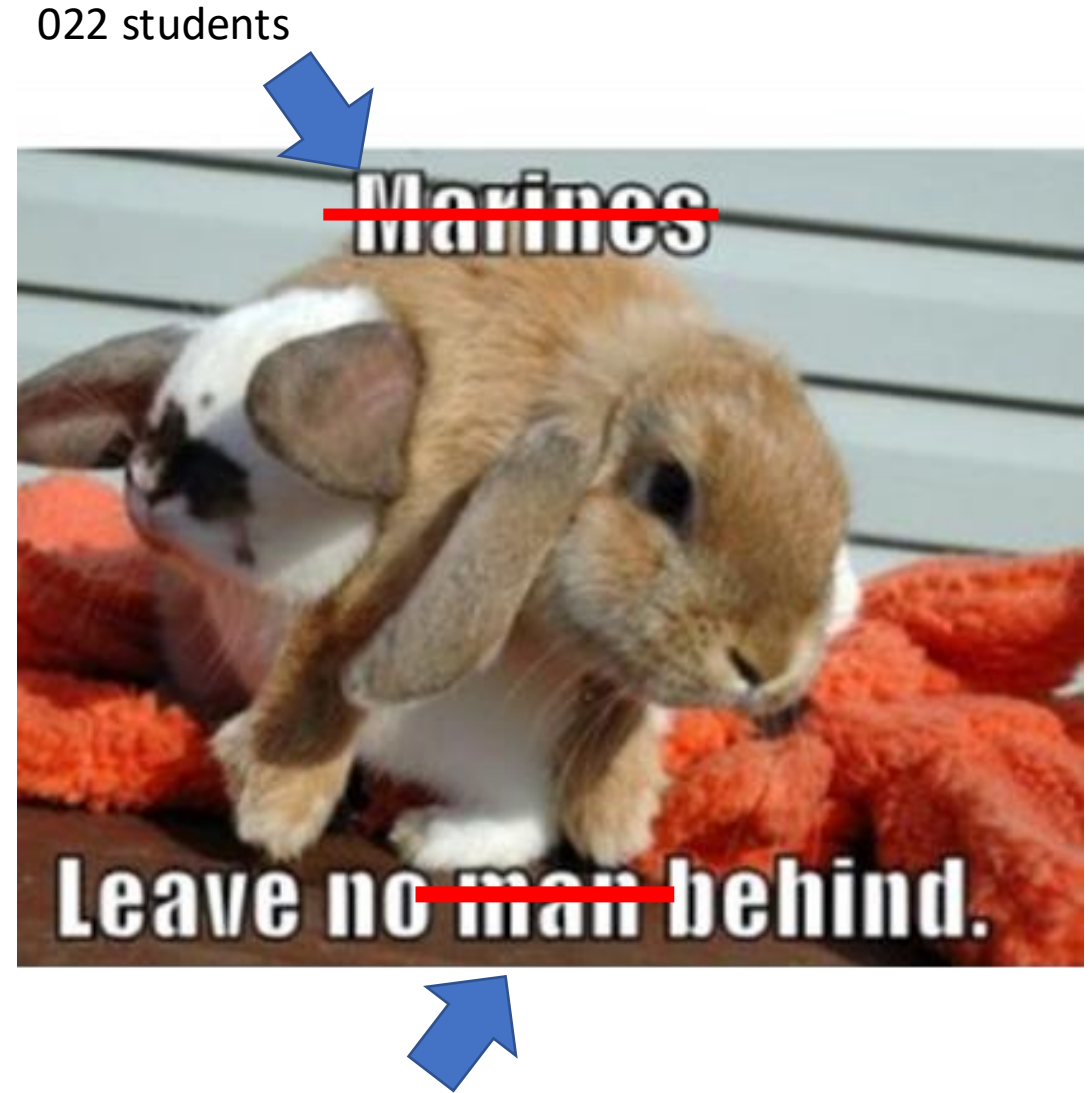
Expression	Evaluates to
<code>static_cast<int>(7.9)</code>	7
<code>static_cast<int>(3.3)</code>	3
<code>static_cast<double>(25)</code>	25.0
<code>static_cast<double>(5 + 3)</code>	= <code>static_cast<double>(8)</code> = 8.0
<code>static_cast<double>(15) / 2</code>	= 15.0 / 2 (because <code>static_cast<double>(15)</code> = 15.0) = 15.0 / 2.0 = 7.5
<code>static_cast<double>(15/2)</code>	= <code>static_cast<double>(7)</code> (because <code>15 / 2 = 7</code>) = 7.0
<code>static_cast<int>(7.8 + static_cast<double>(15)/2)</code>	= <code>static_cast<int>(7.8 + 7.5)</code> = <code>static_cast<int>(15.3)</code> = 15
<code>static_cast<int>(7.8 + static_cast<double>(15/2))</code>	= <code>static_cast<int>(7.8 + 7.0)</code> = <code>static_cast<int>(14.8)</code> = 14

PAUSING FOR A SECOND

- There are aspects of the rest of this talk (and of this course as a whole) that are COMPLICATED
- No, like seriously, it's complicated. Back in undergrad, it took me a little while to get some of this stuff. When I did, I felt a bit like I had just seen the matrix.
- It is review for some of you. It is definitely *not* review for others.
- I don't expect you to get all of this today. I'm introducing it now so that you'll have a better shot at getting it by the end of the semester.

WHAT TO DO WHEN YOU'RE AHEAD OF YOUR CLASSMATES

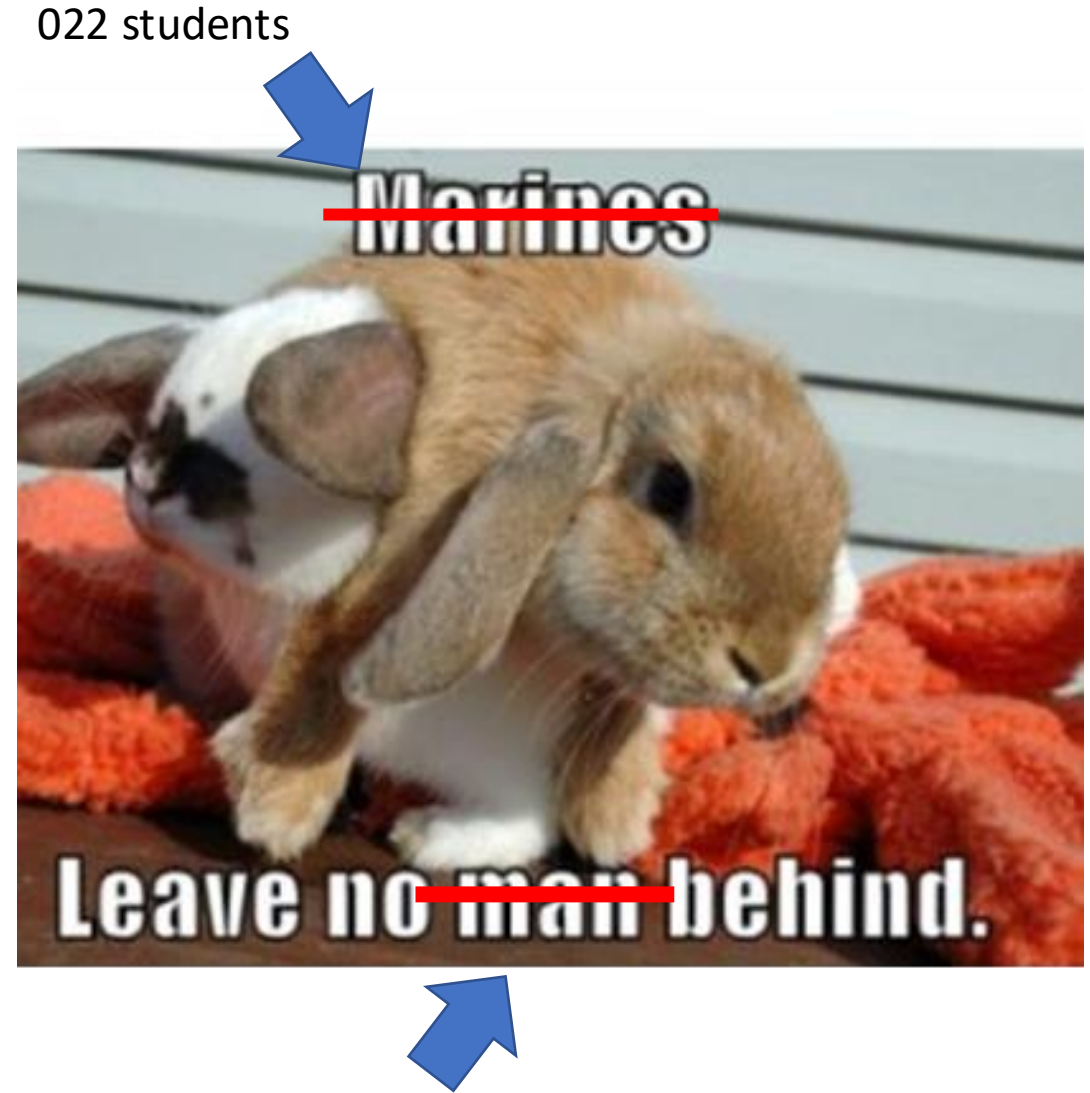
- Say you're like "Karel was super easy and I get this C++ stuff completely"
- Make sure your neighbors (and other classmates) get the material too! The best way to learn something better is to teach it to someone else
- (as I learned when I started teaching)
- ☆ Help your comrades ☆



man, or woman, or non-binary/genderqueer person

WHAT TO DO WHEN YOU'RE BEHIND YOUR CLASSMATES

- Say you're thinking "I don't understand Karel and C++ frightens me and I don't know what to do"
- Ask your neighbors for help!
- I'm about to make everyone promise to help their neighbors whenever they can.
- ☆ Get help from your comrades ☆

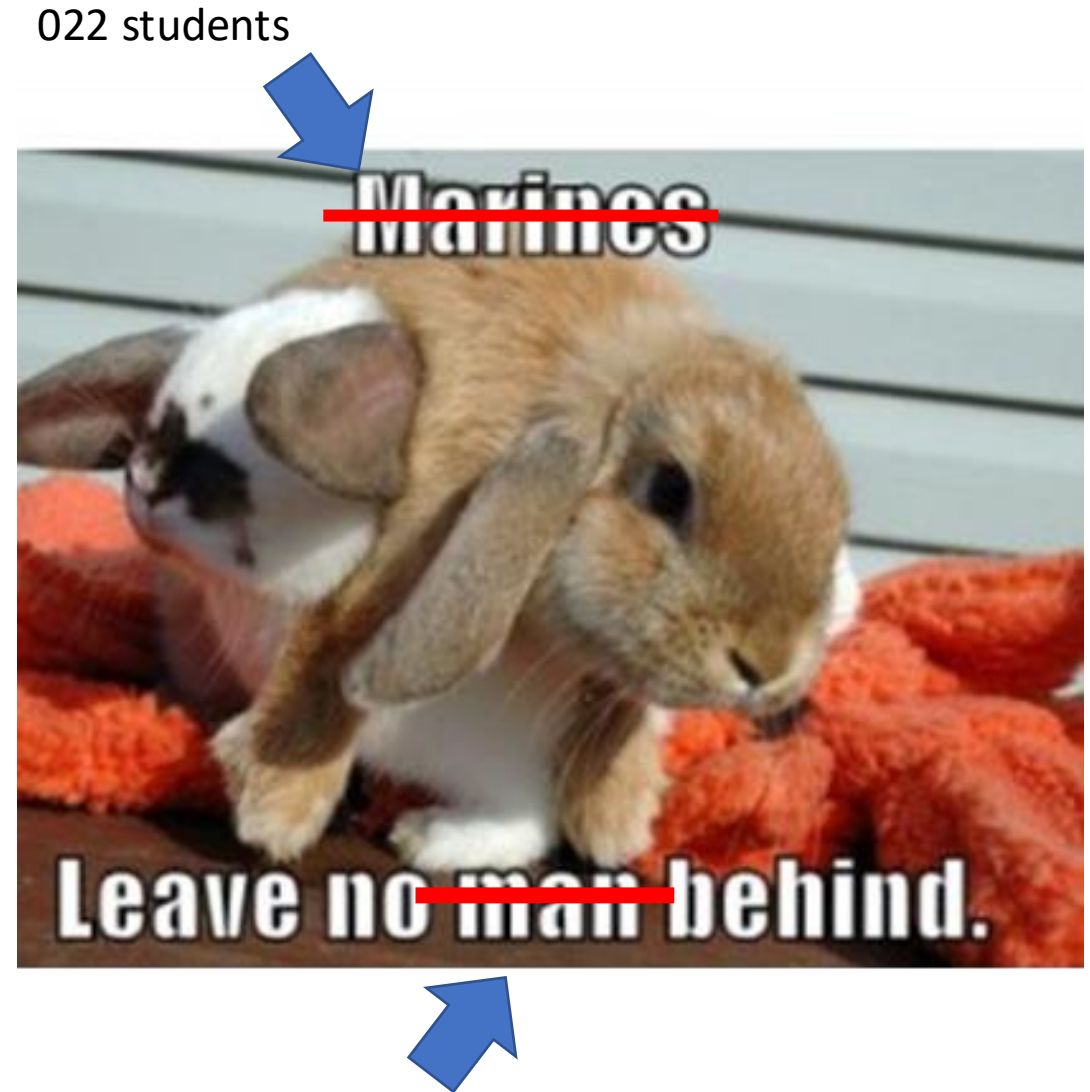


man, or woman, or non-binary/genderqueer person

rules for sharing

- It is **awesome** to help your classmates with how to design their algorithms.
- It is **awesome** to help your classmates with how C++ syntax works
- It is **not awesome** to show them your code. You're not really helping if you just let people copy off of you.

helping each other is dope



man, or woman, or non-binary/genderqueer person

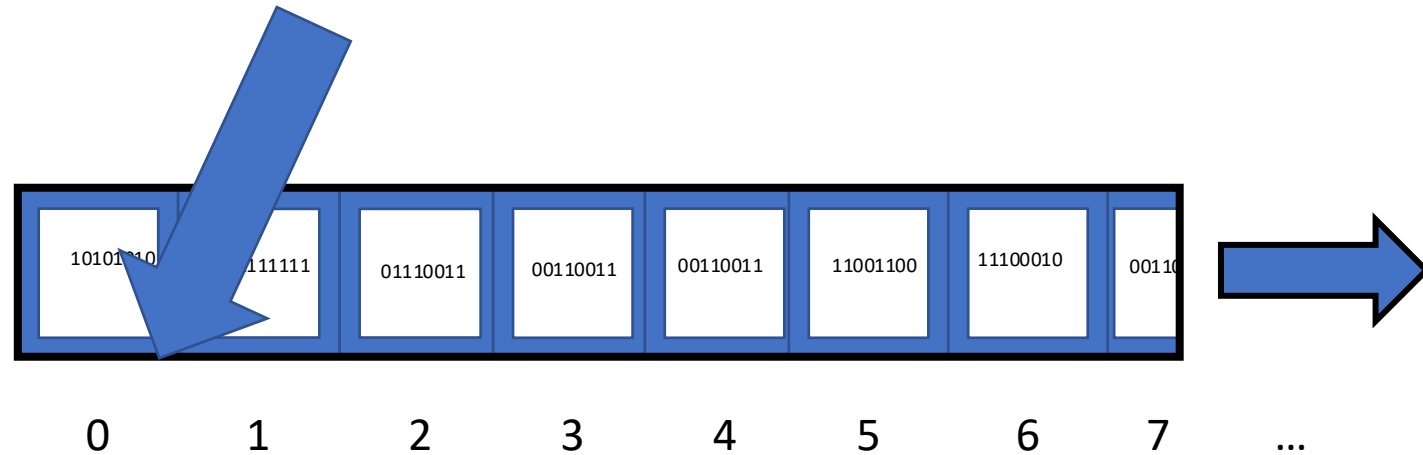
memory – an analogy

You can get the address of an object in memory by using the & operator:

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6
7     int myNumber = 100;
8     int *myPointer = &myNumber;
9     cout << myPointer << endl ;
10
11
12 }
```

a variable that stores an address is called a pointer. we'll talk about pointers in a second

real addresses don't look quite like this, of course...



here's what I get when I run this. this is what addresses typically look like... long hexadecimal numbers

```
myNumber is: 100
the address of myNumber is: 0x7ffee0c1cacc
Bens-MacBook-Air-2:~ benallen$
```

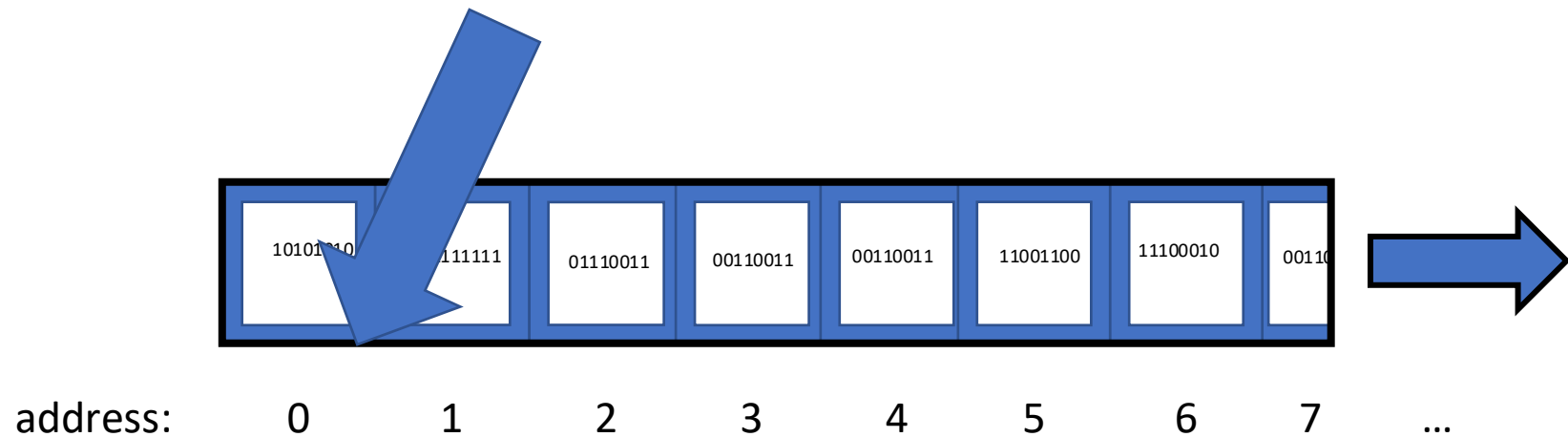
pointers

a pointer is a variable type that stores the address of another variable

you can store the value of an address by using the * indirection operator before the name of a pointer.

once you have an address in the pointer, you can use the * operator again to get what's stored at the address. This is called "dereferencing the pointer"

real addresses don't look quite like this, of course...



```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int myNumber = 100;
7     cout << "myNumber is: " << myNumber << endl;
8     cout << "the address of myNumber is: " << &myNumber << endl;
9
10    int *myPointer = &myNumber;
11
12    cout << "the address stored in myPointer is: " << myPointer << endl;
13
14 }
```

here's what I get when I run this

A screenshot of a terminal window showing the output of the C++ program. The prompt is 'Bens-MacBook-Air-2:~ benallen\$./25addresses'. The output is: 'myNumber is: 100', 'the address of myNumber is: 0x7ffee29acacc', and 'the address stored in myPointer is: 0x7ffee29acacc'. The prompt is followed by a cursor.

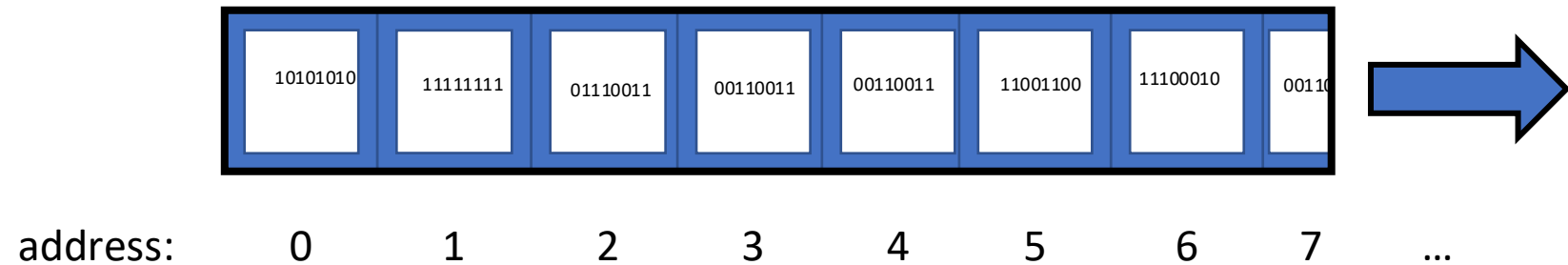
pointers

a pointer is a variable type that stores the address of another variable

declare a pointer by using the name of the type, and then putting * before the name you assign to the pointer.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     int myNumber = 100;
7     cout << "myNumber is: " << myNumber << endl;
8     cout << "the address of myNumber is: " << &myNumber << endl;
9
10    int *myPointer = &myNumber;
11
12    cout << "the address stored in myPointer is: " << myPointer << endl;
13
14 }
```

real addresses don't look quite like this, of course...



here's what I get when I run this

```
Bens-MacBook-Air-2:~ benallen$ ./25addresses
myNumber is: 100
the address of myNumber is: 0x7ffee29acacc
the address stored in myPointer is: 0x7ffee29acacc
Bens-MacBook-Air-2:~ benallen$
```

pointers

pointers are variables just like any other variables, and so you can get their addresses as well:

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     #include <iostream>
7     using namespace std;
8     int main(){
9         int myNumber = 100;
10        cout << "myNumber is: " << myNumber << endl;
11        cout << "the address of myNumber is: " << &myNumber << endl;
12
13        int *myPointer = &myNumber;
14        cout << "the address stored in myPointer is: " << myPointer << endl;
15        int **doubleIndirection = &myPointer;
16        cout << "the address of myPointer is: " << doubleIndirection << endl;
17        cout << "the address of the thing pointed to by myPointer is " << *doubleIndirection << endl;
18        cout << "the value of the thing pointed to by myPointer is " << **doubleIndirection << endl;
19    }
20 }
```

here's what I get when
I run this

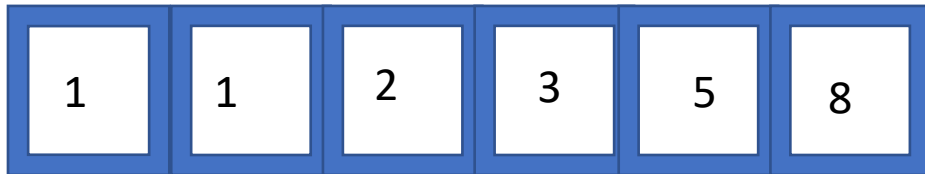


```
[Bens-MacBook-Air-2:~ benallen$ ./25addresses
myNumber is: 100
the address of myNumber is: 0x7ffee54d4acc
the address stored in myPointer is: 0x7ffee54d4acc
the address of myPointer is: 0x7ffee54d4ac0
the address of the thing pointed to by myPointer is 0x7ffee54d4acc
the value of the thing pointed to by myPointer is 100
Bens-MacBook-Air-2:~ benallen$
```

okay so what's an array

arrays are contiguous blocks of memory, starting at the address associated with the name of the array and advancing from there.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int myArray[] = {1, 1, 2, 3, 5, 8} ;
7     cout << myArray[0] << " " << myArray[2] << " " << myArray[4] << endl;
8
9
10 }
11
```



myArray[0] myArray[1] myArray[2] myArray[3] myArray[4] myArray[5]

here's what I get when
I run this

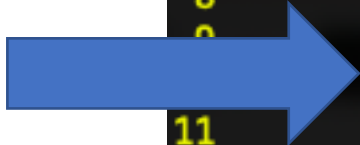
```
Bens-MacBook-Air-2:~ benallen$ ./25addresses
1 2 5
Bens-MacBook-Air-2:~ benallen$
```

arrays

arrays are contiguous blocks of memory, starting at the address associated with the name of the array and advancing from there.

this is maybe
the weirdest
thing I'll show
you all semester.

if you don't get
it, that's fine...
you will by
about seven
weeks from now



```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int myArray[] = {1, 1, 2, 3, 5, 8} ;
7     int a = 5;
8
9     cout << myArray[a] << endl;
10    cout << a[myArray] << endl;
11
12 }
13
14
```

here's what I get when
I run this



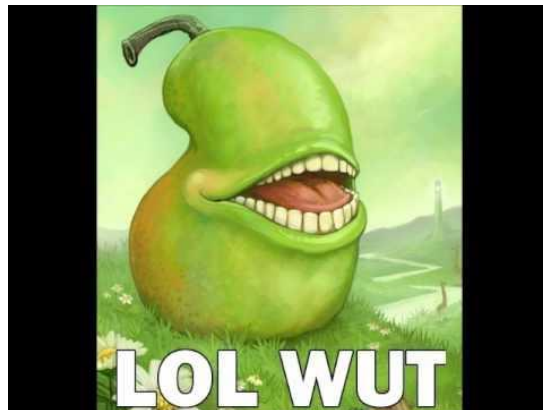
```
[Bens-MacBook-Air-2:~ benallen$ ./25addresses
8
8
Bens-MacBook-Air-2:~ benallen$
```

arrays

arrays are contiguous blocks of memory, starting at the address associated with the name of the array and advancing from there.

this is maybe the weirdest thing I'll show you all semester.

if you don't get it, that's fine... you will by about seven weeks from now



```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(){
6     int myArray[] = {1, 1, 2, 3, 5, 8} ;
7     int a = 5;
8
9     cout << myArray[a] << endl;
10    cout << a[myArray] << endl;
11
12 }
13
14
```

here's what I get when I run this

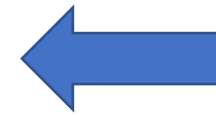
```
Bens-MacBook-Air-2:~ benallen$ ./25addresses
8
8
Bens-MacBook-Air-2:~ benallen$
```

arrays under the hood

- arrays names refer to addresses.
- I'm going to repeat this again: array names refer to *addresses*.

```
#!/bin/bash [running]
1 #include <iostream>
2
3 using namespace std;
4
5 int main(int argc, char* argv[]){
6     int myArray[] = {1, 2, 3, 4, 5};
7     cout << myArray << endl;
8     // cout << *myArray << endl;
9     // cout << *myArray + 4 << endl;
10
11 }
12
13
```

when I run this:



I get this:



it's an address!

```
bash-3.2$ g++ -o arrays arrays.cpp
bash-3.2$ ./arrays
0x7ffee5529b0
bash-3.2$
```

arrays under the hood

- if you dereference an array using the `*` operator, you get the first value in the array
- ... cool, right?

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(int argc, char* argv[]){
6     int myArray[] = {1, 2, 3, 4, 5};
7     cout << myArray << endl;
8     cout << *myArray << endl;
9     cout << *myArray + 4 << endl;
10
11 }
12
13
```

when I run this:

I get this:

```
Bens-MacBook-Air-2:~ benallen$ ./arrays
0x7ffeec5cdac0
1
Bens-MacBook-Air-2:~ benallen$
```

it's the value at `myArray[0]`

arrays under the hood

- you can do math with addresses. $*(myArray + 4)$ is the same value as `myArray[4]`
- ... weird, right?

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main(int argc, char* argv[]){
6     int myArray[] = {1, 2, 3, 4, 5};
7     cout << myArray << endl;
8     cout << *myArray << endl;
9     cout << *myArray + 4 << endl;
10
11 }
```

this is called pointer math.
you can do deep magic with it...
but use it sparingly. I'm not showing it to
you so that you can use it all the time. I'm
showing it to you so that you know what
arrays are under the hood

when I run this:



I get this:

```
Bens-MacBook-Air-2:~ benallen$ ./arrays
0x7ffedef6ac0
1
5
```

it's the value at `myArray[4]`

Putting it all together

- Say you've got the following array:

```
int myArray[4] = {2, 3, 23, 137};
```
- `myArray[0]` is the *value* of the first element of the array: 2
- `myArray` by itself is the *address* of the first element of the array (some long hexadecimal number)
- `myArray + 1` is the *address* of the second element of the array
- `*(myArray + 1)` is the *value* of the second element of the array: 3
- `*(1 + myArray)` is also the value of the second element of the array, because addition is commutative
- Therefore... and this is something you should literally never do...
- `1[myArray]` is equivalent to `myArray[1]`.
 - Most languages don't let you do weird stuff like this. C++ is like "LOL, okay, you want to do that weird thing? Go ahead!" This is indicative of the design of C++. It can be incredibly powerful, because it lets you do weird, useful things... but it also can be incredibly frustrating, because it doesn't stop you from accidentally doing weird, useless things.

expectations

- You will not be expected to do fancy stuff with pointers and addresses (or really, to use pointers at all) until a few weeks into the class. don't freak out if you're confused right now.